Knapsack using Dynamic Programming

```
def knapsack(weights, values, capacity):
    n = len(weights)

    # Create DP table
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

    # Fill the DP table
    for i in range(1, n + 1):
        for w in range(1, capacity + 1):

            # If item weight is less than or equal to capacity
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                )

            # Otherwise, don't take the item
            else:
                dp[i][w] = dp[i - 1][w]

    return dp[n][capacity]


# User input
n = int(input("Enter number of items: "))

weights = []
values = []

for i in range(n):
    weight = int(input(f"Enter weight of item {i + 1}: "))
    value = int(input(f"Enter value of item {i + 1}: "))

    weights.append(weight)
    values.append(value)

capacity = int(input("Enter knapsack capacity: "))

# Calculate maximum value
result = knapsack(weights, values, capacity)

print("Maximum value =", result)
```

```
Enter number of items: 5
Enter weight of item 1: 23
Enter value of item 1: 56
Enter weight of item 2: 2
Enter value of item 2: 45
Enter weight of item 3: 21
Enter value of item 3: 56
Enter weight of item 4: 22
Enter value of item 4: 46
Enter weight of item 5: 43
Enter value of item 5: 23
Enter knapsack capacity: 6
Maximum value = 45
```